

Theory of Automata

CS 3005

Lecture 15

Mahzaib Younas

Lecturer, Department of Computer Science
FAST NUCES CFD

Context Free Grammars
for
Regular Languages

Semi Word

- For a given CFG, semiword is a string of terminals (may be none) concatenated with exactly one non-terminal (on the right).
- In general semiword has the shape

(terminal) (terminal)...(terminal) (Non-Terminal)

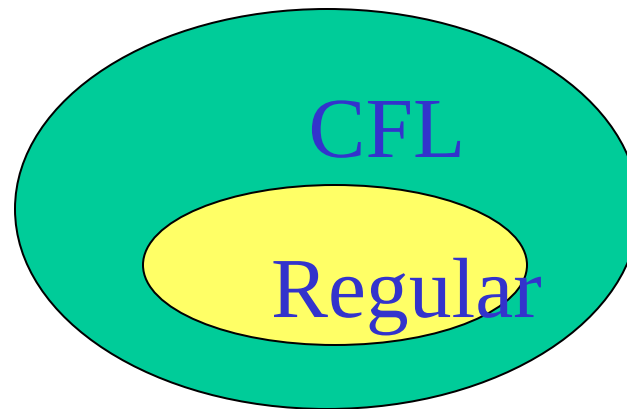
e.g. aaaX abcY bbY

A word is a string of terminals only (zero or more terminals)

Regular Grammar

Given an FA, there is a CFG that generates exactly the language accepted by the FA.

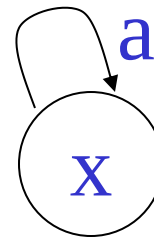
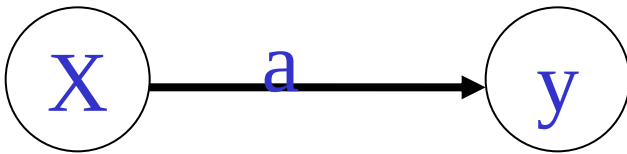
- In other words, all regular languages are CFLs



Creating a CFG from an FA

Step-1 The Non-terminals in CFG will be all names of the states in the FA with the start state renamed S.

Step-2 For every edge



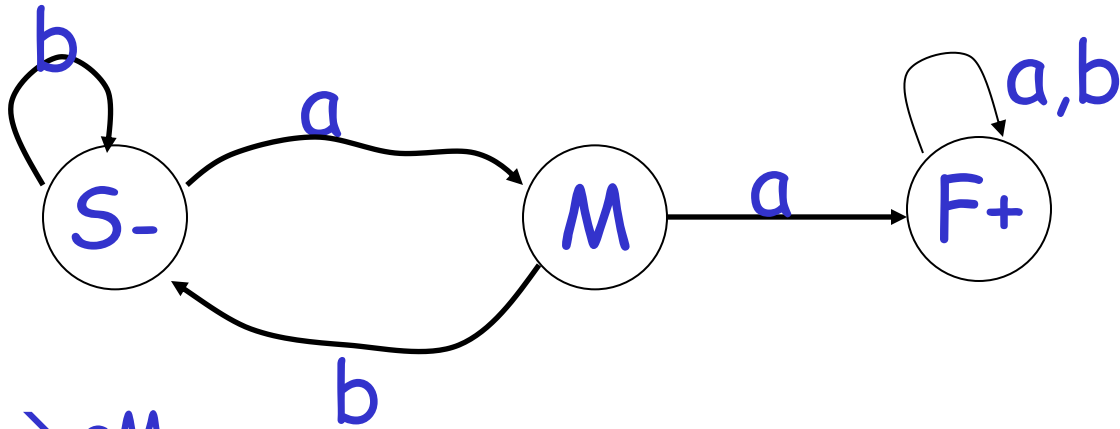
Create productions $X \rightarrow aY$ or $X \rightarrow aX$

Do the same for b-edges

Step-3 For every final-state X, create the production

$$X \rightarrow \Lambda$$

Example



$S \rightarrow aM$

$S \rightarrow bS$

$M \rightarrow aF$

$M \rightarrow bS$

$F \rightarrow aF$

$F \rightarrow bF$

$F \rightarrow \Lambda$

Note: It is not necessary that each CFG has a corresponding FA. But each FA has an equivalent CFG.

Regular Grammar

Theorem 22:

If all the productions in a given CFG fit one of the two forms:

Non-terminal $\rightarrow$ semiword
or Non-terminal $\rightarrow$ word

(Where the word may be a Λ or string of terminal),
then the language generated by the CFG is
Regular.

Proof:

For a CFG to be regular is by constructing a TG
from the given CFG.

Proof contd.

- Let us consider a general CFG in this form

$$N_1 \rightarrow w_1 N_2$$

$$N_7 \rightarrow w_{10}$$

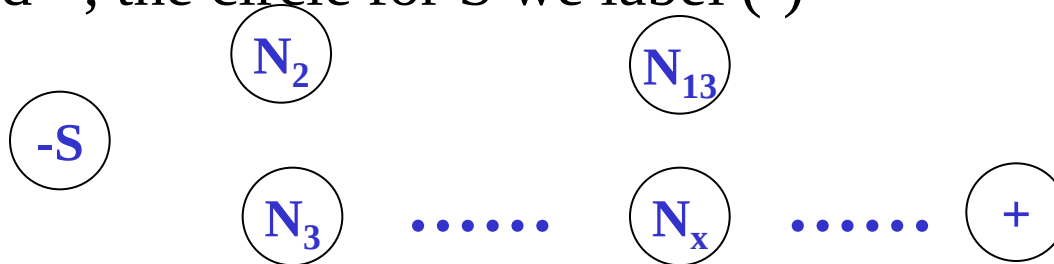
$$N_1 \rightarrow w_2 N_3$$

$$N_{18} \rightarrow w_{23}$$

$$N_2 \rightarrow w_3 N_4$$

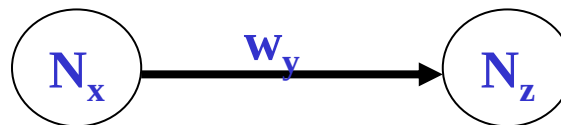
Where N's are non-terminal and w's are the string of terminal and part $w_y N_z$ are semiwords.

Let $N_1 = S$. Draw a small circle for each N and one extra circle labelled +, the circle for S we label (-)



Proof contd.

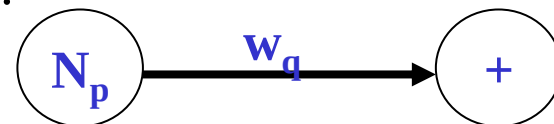
- For each production of the form $N_x \rightarrow w_y N_z$, draw a directed edge from state N_x to N_z with label w_y .
- If $N_x = N_z$, the path is a loop
- For every production of the form $N_p \rightarrow w_q$, draw a directed edge from N_p to $+$ and label it with w_q even if $w_q = \Lambda$.



- Any path in TG form – to $+$ corresponds to a word in the language of TG (by concatenating symbols) and simultaneously corresponds to sequence of productions on the CFG generating words.
- Conversely every production of the word in the CFG:

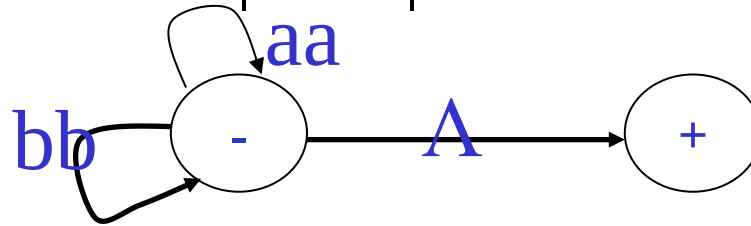
$S \rightarrow wN \rightarrow wwN \rightarrow wwwN \rightarrow \dots \rightarrow wwwww$

Corresponds to a path in this TG.



Example

- Consider the CFG $S \rightarrow aaS \mid bbS \mid \Lambda$



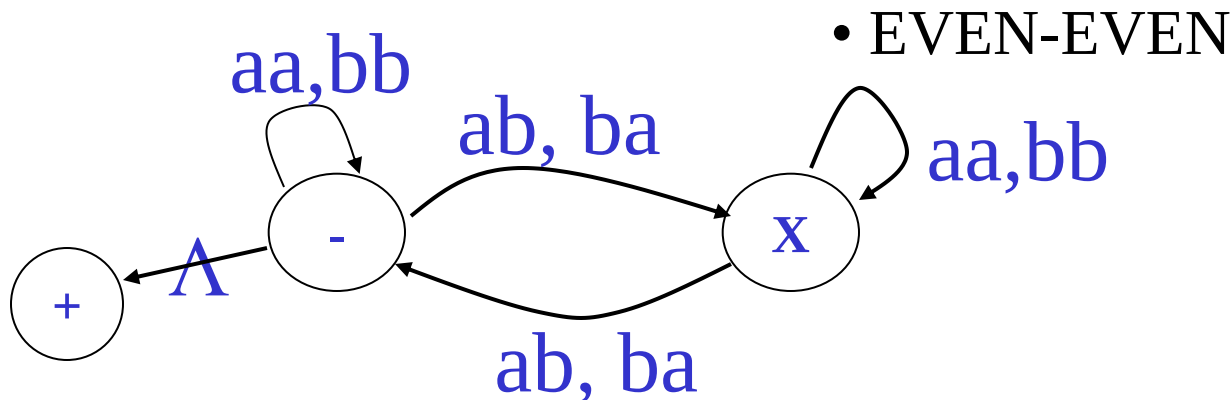
- The regular expression is given by $(aa + bb)^*$.

- Consider the CFG

$S \rightarrow aaS \mid bbS \mid abX \mid baX \mid \Lambda$

- Language accepted?

$X \rightarrow aaX \mid bbX \mid abS \mid baS$



- EVEN-EVEN

Killing Λ -Productions

Λ -Productions and **nullable** variables

In a given CFG, we call a non-terminal N **nullable** if there is a production $N \rightarrow \Lambda$, or there is a derivation that starts at N and lead to a Λ .

$$N \rightarrow \dots\dots\dots \rightarrow \Lambda$$

- Λ -Productions are undesirable.
- We can replace Λ -production with appropriate non- Λ productions.

Theorem 23

- If L is CFL generated by a CFG having Λ -productions, then there is a different CFG that has no Λ -production and still generates either the whole language L (if L does not include Λ) or else generate the language of all the words in L other than Λ .
- Replacement Rule.
 1. Delete all Λ -Productions.
 2. Add the following productions:

For every production of the $X \rightarrow \text{old string}$ Add new production of the form $X \rightarrow ..$, where right side will account for every modification of the old string that can be formed by deleting all possible subsets of null-able Non-Terminals, except that we do not allow $X \rightarrow \Lambda$, to be formed if all the character in old string are null-able

- Delete all the Null productions and add new productions e.g.
- consider the productions of a certain CFG

$$X \rightarrow aNbNa,$$

$$N \rightarrow \Lambda,$$

Delete the production $N \rightarrow \Lambda$ and using the

- production $X \rightarrow aNbNa$, add the new productions $X \rightarrow aNba$, $X \rightarrow abNa$ and $X \rightarrow aba$
- Thus the new CFG will contain the productions
 $X \rightarrow aNba|abNa|aba|aNbNa$
- Note: It is to be noted that $X \rightarrow aNbNa$ will still be included in the new CFG.

Example

Consider the CFG

$S \rightarrow a \mid Xb \mid aYa$

$X \rightarrow Y \mid \Lambda$

$Y \rightarrow b \mid X$

X and Y are nullable

X as $X \rightarrow \Lambda$

Y as $Y \rightarrow X \rightarrow \Lambda$

So the new CFG is

$S \rightarrow a \mid Xb \mid aa \mid aYa \mid b$

$X \rightarrow Y$

$Y \rightarrow b \mid X$

Example

Consider the CFG

$$S \rightarrow Xa$$
$$X \rightarrow aX \mid bX \mid \Lambda$$

So the new CFG is

$$S \rightarrow a \mid Xa$$
$$X \rightarrow aX \mid bX \mid a \mid b$$

Example

$S \rightarrow XY$

$X \rightarrow Zb$

$Y \rightarrow bW$

$Z \rightarrow AB$

$W \rightarrow Z$

$A \rightarrow aA \mid bA \mid \Lambda$

$B \rightarrow Ba \mid Bb \mid \Lambda$

- Null-able Non-terminals are?
- A, B, Z and W

Example Contd.

$S \rightarrow XY$

$X \rightarrow Zb$

$Y \rightarrow bW$

$Z \rightarrow AB$

$W \rightarrow Z$

$A \rightarrow aA \mid bA \mid \Lambda$

$B \rightarrow Ba \mid Bb \mid \Lambda$

So the new CFG is

$S \rightarrow XY$

$X \rightarrow Zb \mid b$

$Y \rightarrow bW \mid b$

$Z \rightarrow AB \mid A \mid B$

$W \rightarrow Z$

$A \rightarrow aA \mid bA \mid a \mid b$

$B \rightarrow Ba \mid Bb \mid a \mid b$

MEMBERSHIP

The Total Language Tree

- It is possible to depict the generation of all the words in the language of a CFG simultaneously in one big (possibly infinite) tree.

Definition:

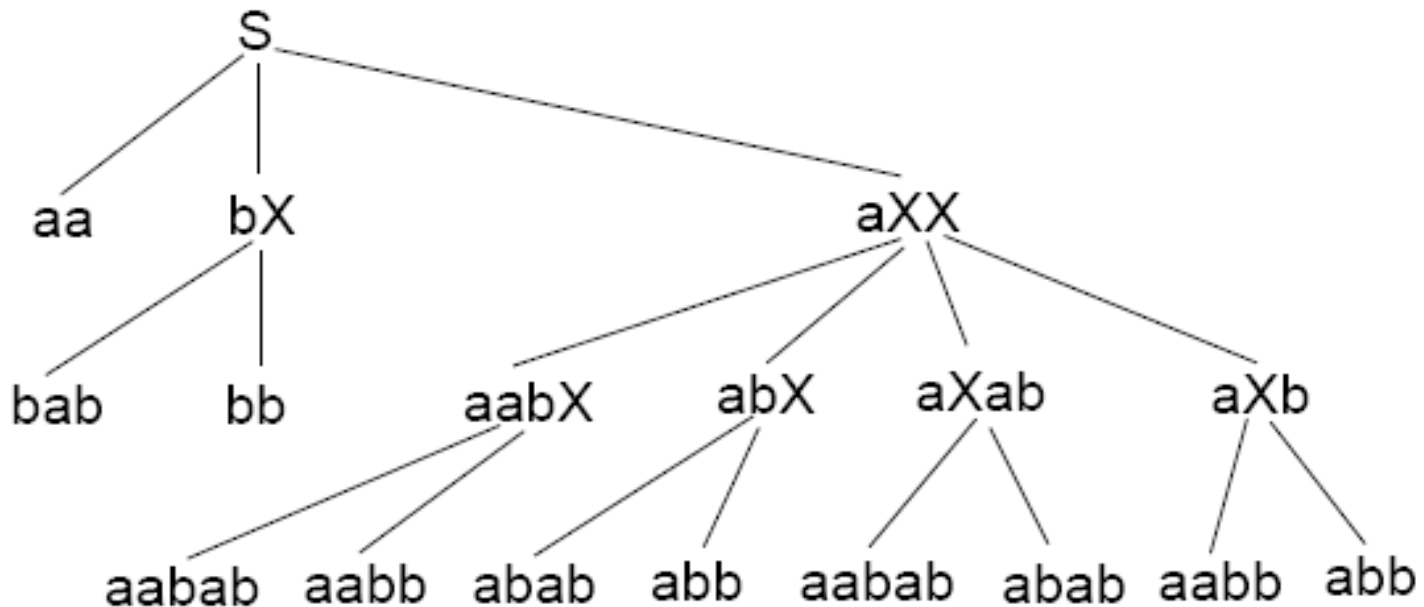
- For a given CFG, we define a tree with the start symbol S as its root and whose nodes are working strings of terminals and nonterminals. The descendants of each node are all the possible results of applying every applicable production to the working string, one at a time. A string of all terminals is a terminal node in the tree. The resultant tree is called the **total language tree** of the CFG.

Example

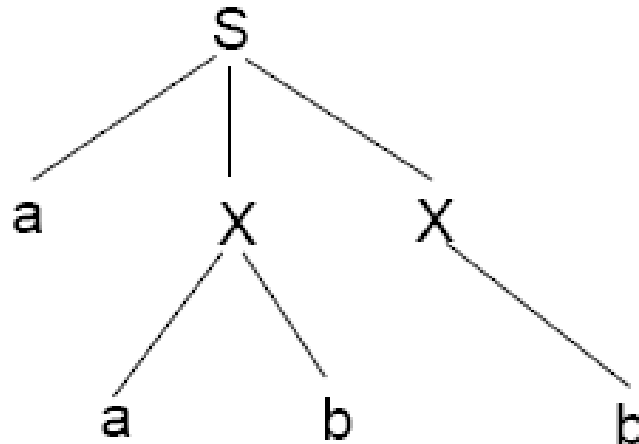
- Consider the CFG:

$$S \rightarrow aa \mid bX \mid aXX$$
$$X \rightarrow ab \mid b$$

- The total language tree is



- The above total language has only 7 different words.
- Four of its words (abb, aabb, abab, aabab) have two different derivations because they appear as terminal nodes in two different places.
- However, these words are NOT generated by two different derivation trees. Hence, the CFG is unambiguous. For example,



Example

- Consider the CFG:

$$S \rightarrow aSb \mid bS \mid a$$

- The language of this CFG is infinite, so is the total language tree: The tree may get arbitrary wide as well as infinitely long.
- Can you draw the beginning part of this total language tree?

Linear Grammars

Grammars with
at most one variable at the right side
of a production

Examples:

$S \rightarrow aSb$	$S \rightarrow Ab$
$S \rightarrow \lambda$	$A \rightarrow aAb$
	$A \rightarrow \lambda$

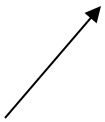
A Non-Linear Grammar

Grammar G :

$$S \rightarrow SS$$
$$S \rightarrow \lambda$$
$$S \rightarrow aSb$$
$$S \rightarrow bSa$$

$$L(G) = \{w : n_a(w) = n_b(w)\}$$

Number of a in string w



Another Linear Grammar

Grammar G :

$$S \rightarrow A$$
$$A \rightarrow aB \mid \lambda$$
$$B \rightarrow Ab$$

$$L(G) = \{a^n b^n : n \geq 0\}$$

Right-Linear Grammars

All productions have form: $A \rightarrow xB$

or

$$A \rightarrow x$$



Example: $S \rightarrow abS$

$$S \rightarrow a$$

string of
terminals

Left-Linear Grammars

All productions have form: $A \rightarrow Bx$

or

$$A \rightarrow x$$



Example: $S \rightarrow Aab$

$$A \rightarrow Aab \mid B$$

$$B \rightarrow a$$

string of
terminals

Regular Grammars

Regular Grammars

A regular grammar is any
right-linear or left-linear grammar

Examples:

G_1

$S \rightarrow abS$

$S \rightarrow a$

G_2

$S \rightarrow Aab$

$A \rightarrow Aab \mid B$

$B \rightarrow a$

Observation

Regular grammars generate regular languages

Examples:

G_1

$S \rightarrow abS$

$S \rightarrow a$

$L(G_1) = (ab)^* a$

G_2

$S \rightarrow Aab$

$A \rightarrow Aab \mid B$

$B \rightarrow a$

$L(G_2) = aab(ab)^*$

Regular Grammars
Generate
Regular Languages

Theorem

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Grammars} \end{array} \right\} = \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Theorem - Part 1

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Grammars} \end{array} \right\} \subseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Any regular grammar generates
a regular language

Theorem - Part 2

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Grammars} \end{array} \right\} \supseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Any regular language is generated
by a regular grammar

Proof - Part 1

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Grammars} \end{array} \right\} \subseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

The language $L(G)$ generated by
any regular grammar G is regular

The case of Right-Linear Grammars

Let G be a right-linear grammar

We will prove: $L(G)$ is regular

Proof idea: We will construct NFA M
with $L(M) = L(G)$

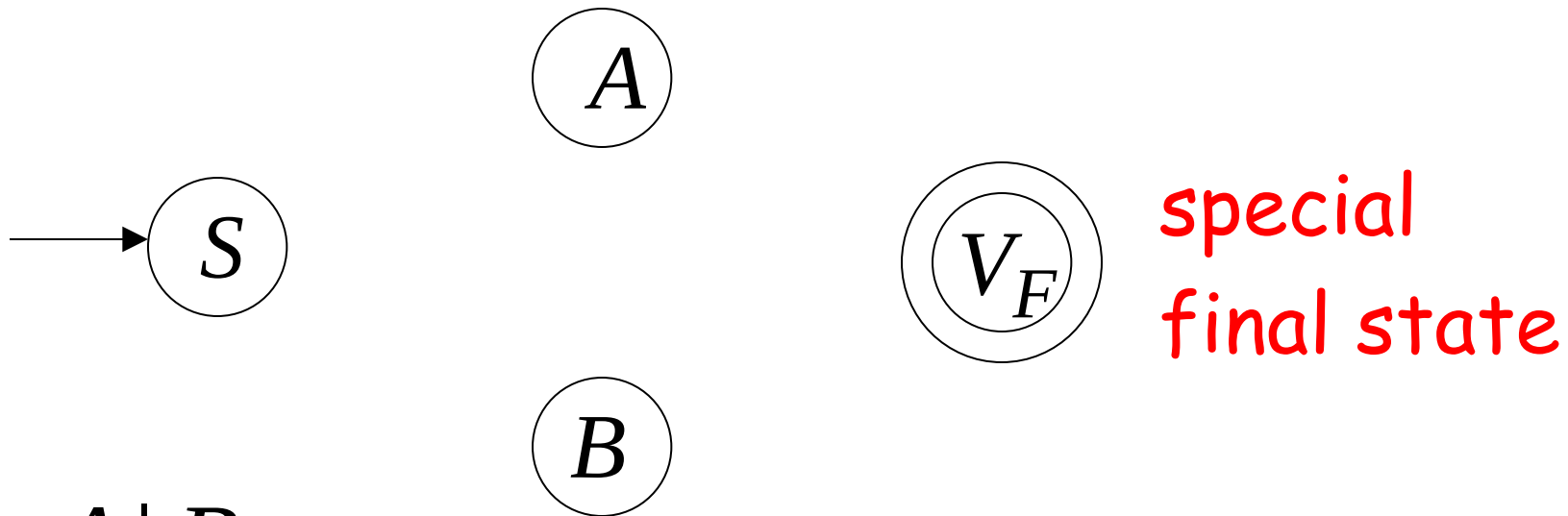
Grammar G is right-linear

Example: $S \rightarrow aA \mid B$

$A \rightarrow aa B$

$B \rightarrow b B \mid a$

Construct NFA M such that
every state is a grammar variable:

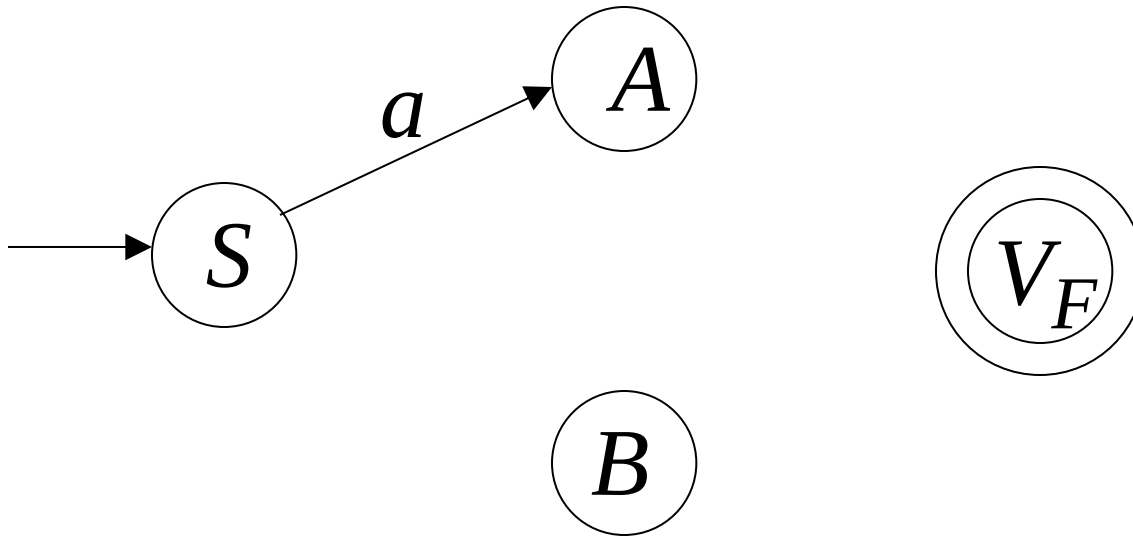


$$S \rightarrow aA \mid B$$

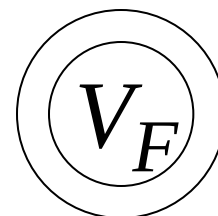
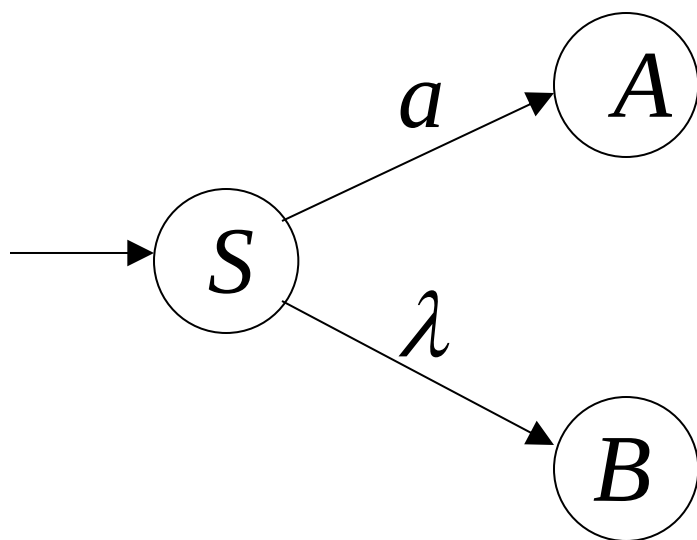
$$A \rightarrow aa B$$

$$B \rightarrow b B \mid a$$

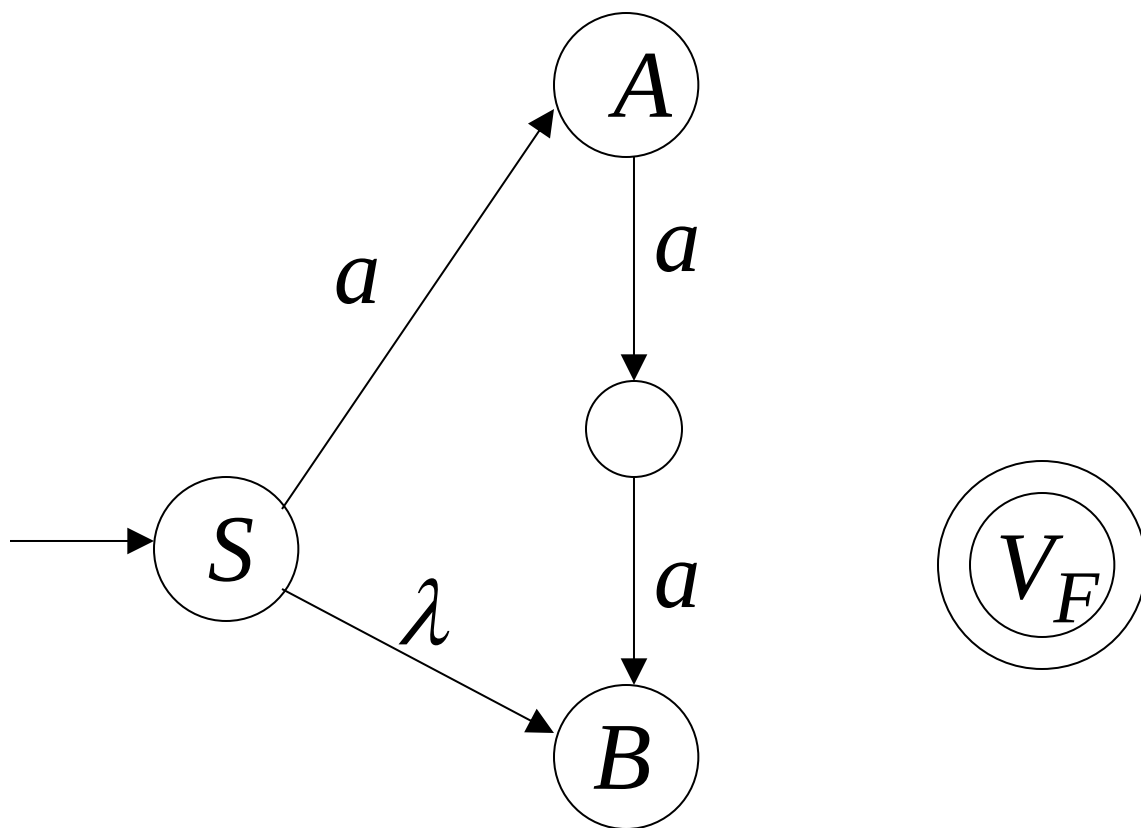
Add edges for each production:



$$S \rightarrow aA$$

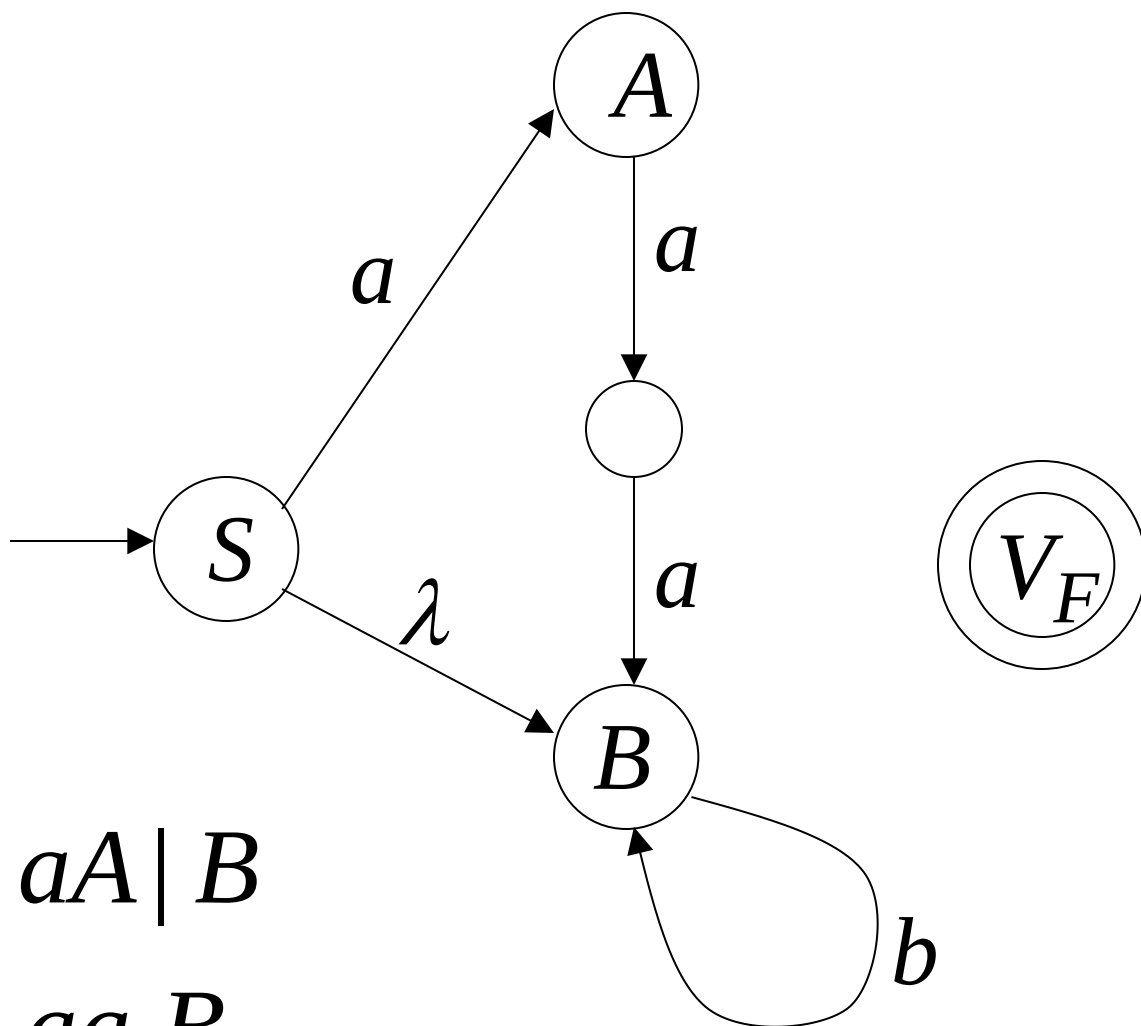


$S \rightarrow aA \mid B$



$S \rightarrow aA \mid B$

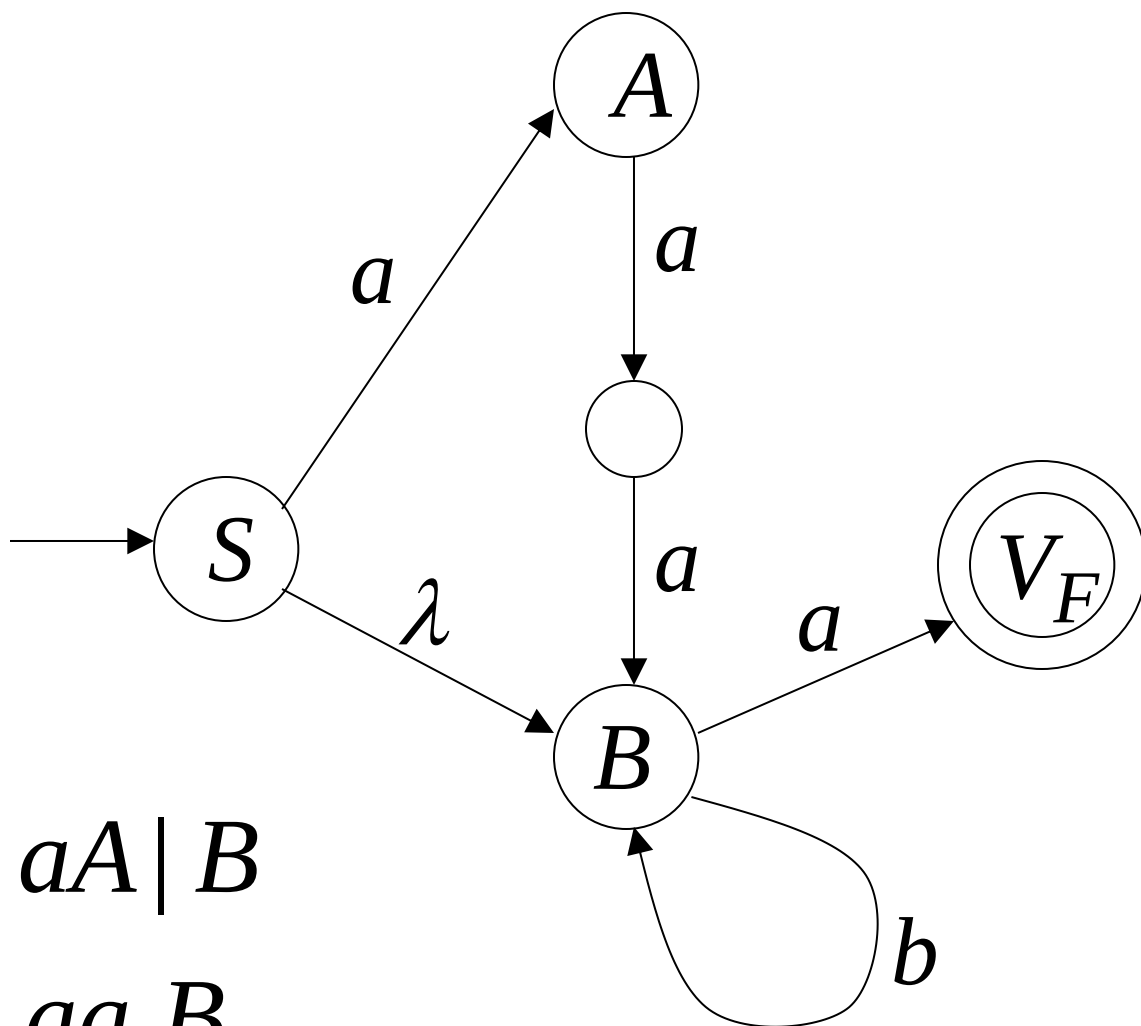
$A \rightarrow aa B$



$S \rightarrow aA \mid B$

$A \rightarrow aa B$

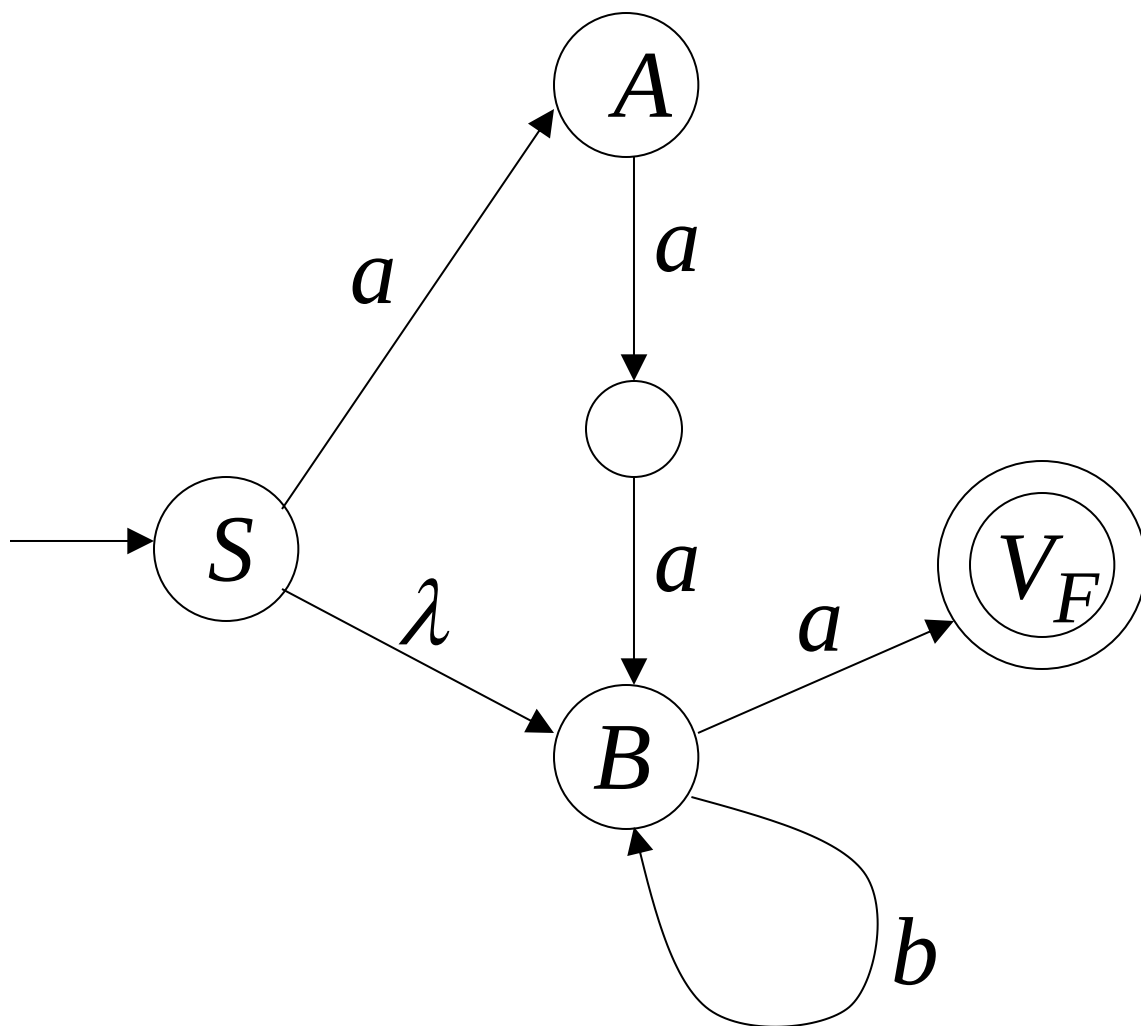
$B \rightarrow bB$



$S \rightarrow aA \mid B$

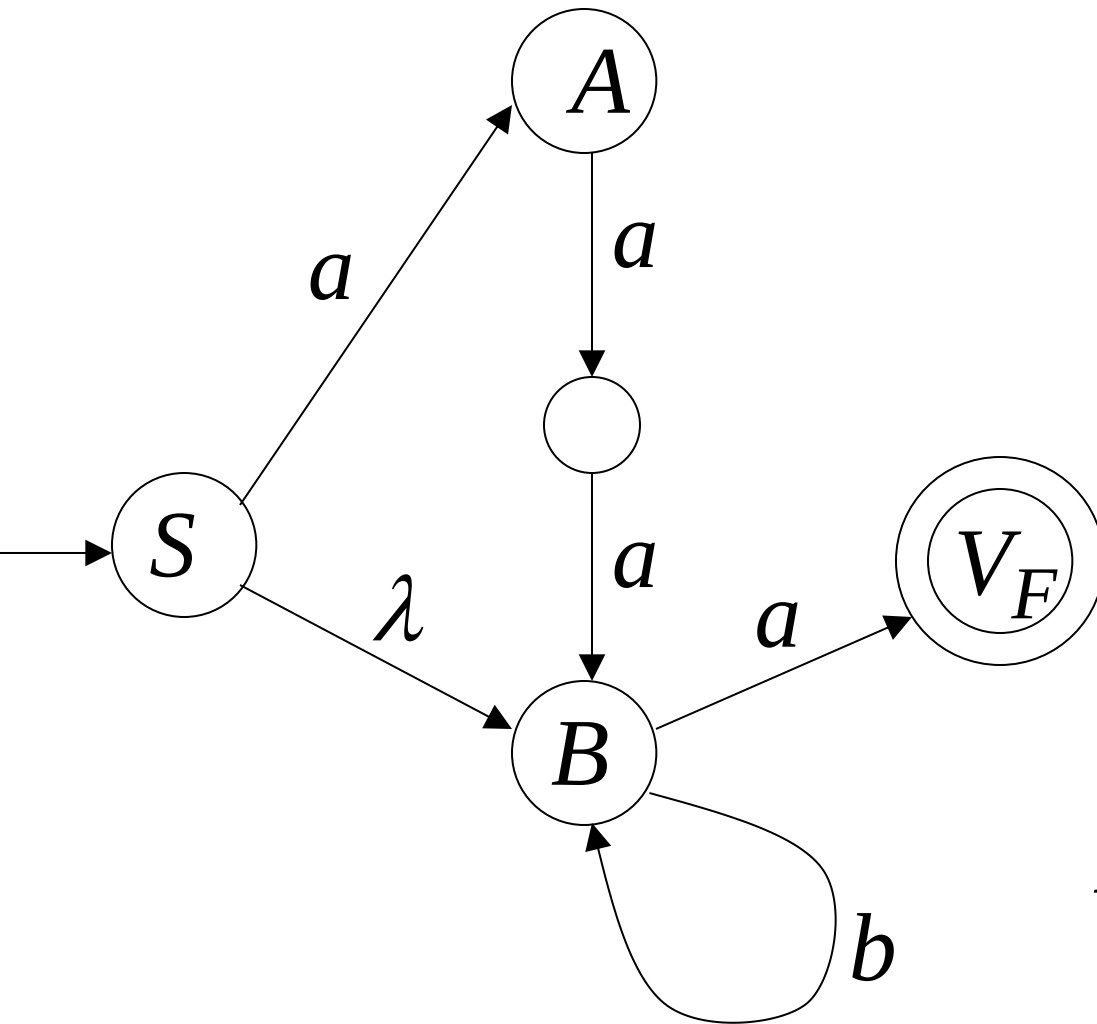
$A \rightarrow aa B$

$B \rightarrow bB \mid a$



$S \Rightarrow aA \Rightarrow aaaS \Rightarrow aaabB \Rightarrow aaaba$

NFA M



Grammar

G

$S \rightarrow aA \mid B$

$A \rightarrow aaB$

$B \rightarrow bB \mid a$

$L(M) = L(G) =$
 $aaaab^*a + b^*a$

In General

A right-linear grammar G

has variables: $V_0, V_1, V_2, \dots$

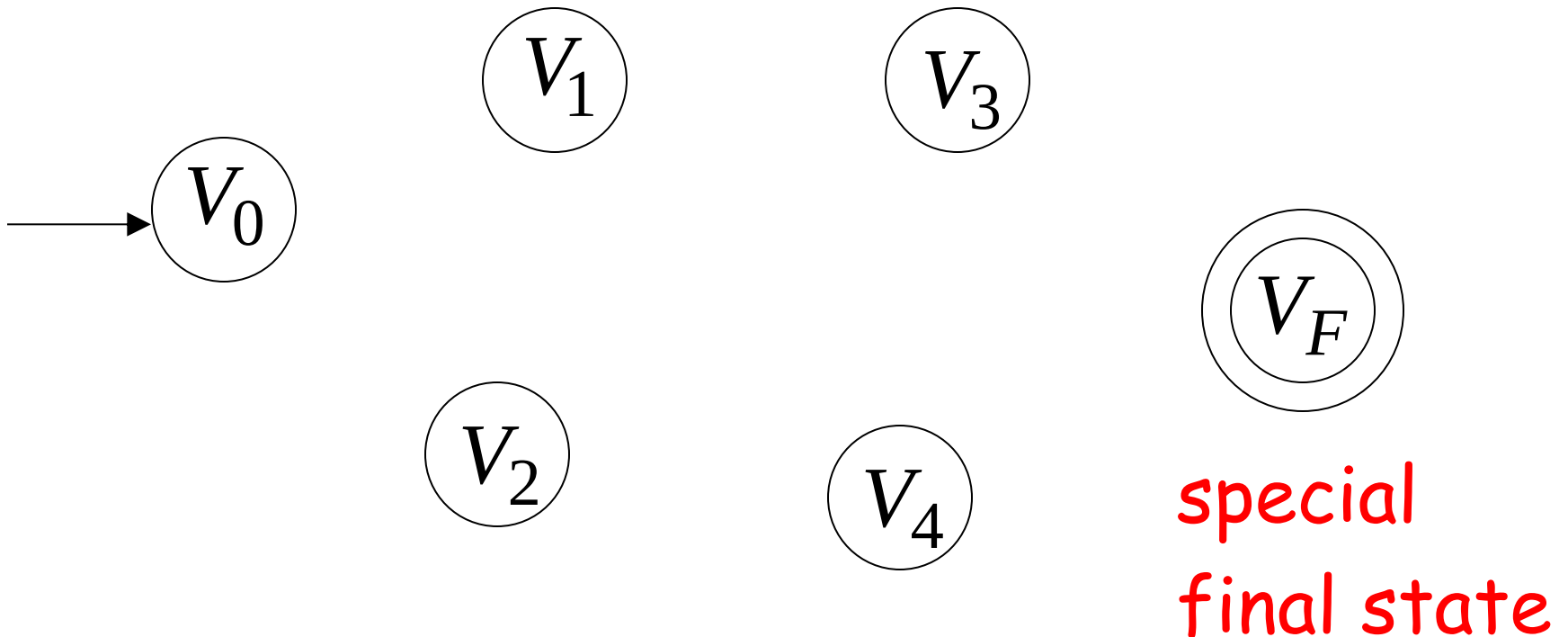
and productions: $V_i \rightarrow a_1 a_2 \cdots a_m V_j$

or

$$V_i \rightarrow a_1 a_2 \cdots a_m$$

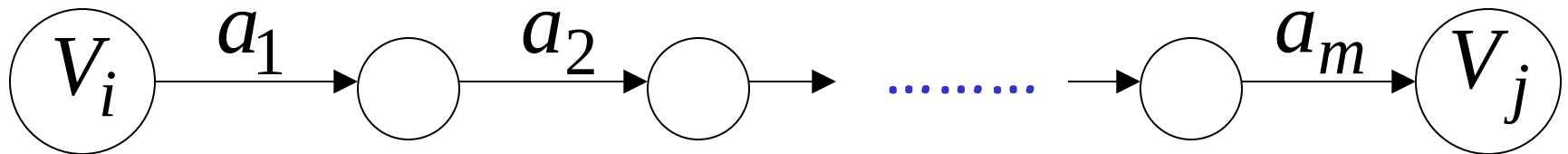
We construct the NFA M such that:

each variable V_i corresponds to a node:



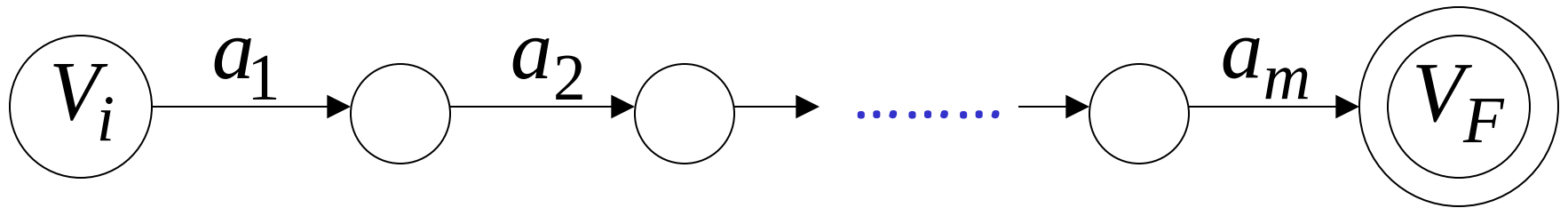
For each production: $V_i \rightarrow a_1 a_2 \cdots a_m V_j$

we add transitions and intermediate nodes



For each production: $V_i \rightarrow a_1 a_2 \cdots a_m$

we add transitions and intermediate nodes



The case of Left-Linear Grammars

Let G be a left-linear grammar

We will prove: $L(G)$ is regular

Proof idea:

We will construct a right-linear grammar G' with $L(G) = L(G')^R$

Since G is left-linear grammar
the productions look like:

$$A \rightarrow Ba_1a_2 \cdots a_k$$

$$A \rightarrow a_1a_2 \cdots a_k$$

Construct right-linear grammar G'

Left
linear

G

$$A \rightarrow Ba_1a_2 \cdots a_k$$

$$A \rightarrow Bv$$



Right
linear

G'

$$A \rightarrow a_k \cdots a_2a_1B$$

$$A \rightarrow v^R B$$

Construct right-linear grammar G'

Left
linear

G

$$A \rightarrow a_1 a_2 \cdots a_k$$

$$A \rightarrow v$$



Right
linear

G'

$$A \rightarrow a_k \cdots a_2 a_1$$

$$A \rightarrow v^R$$

Example

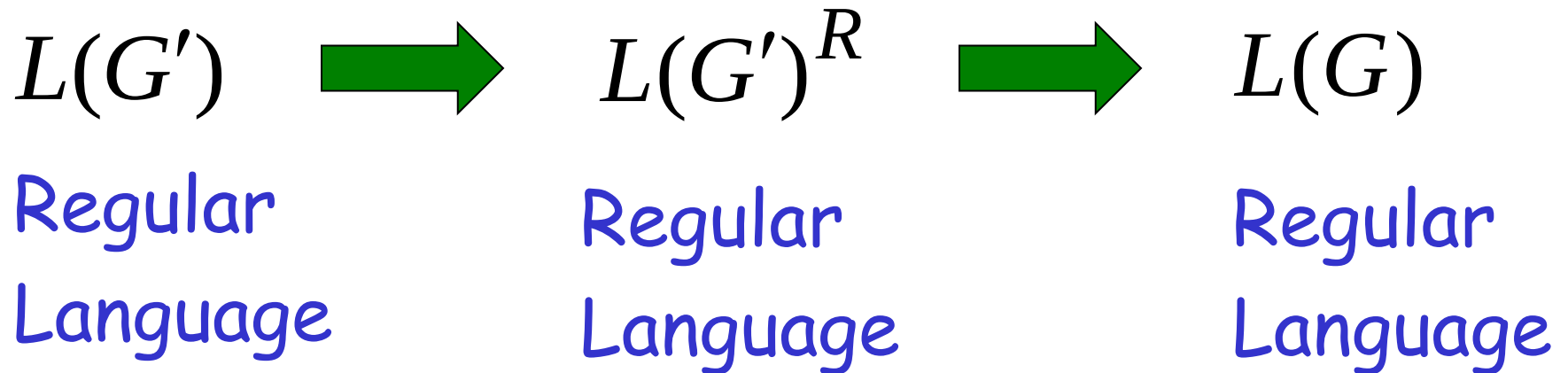
- $A \rightarrow Ba \mid Ab \mid b$
- $B \rightarrow Ca \mid Bb$
- $C \rightarrow Aa \mid Cb$

To convert this Left Linear Grammer First we take it reverse

- $A \rightarrow aB \mid bA \mid b$
- $B \rightarrow aC \mid bB$
- $C \rightarrow aA \mid bC$

It is easy to see that: $L(G) = L(G')^R$

Since G' is right-linear, we have:



Proof - Part 2

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Grammars} \end{array} \right\} \supseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Any regular language L is generated
by some regular grammar G

Any regular language L is generated
by some regular grammar G

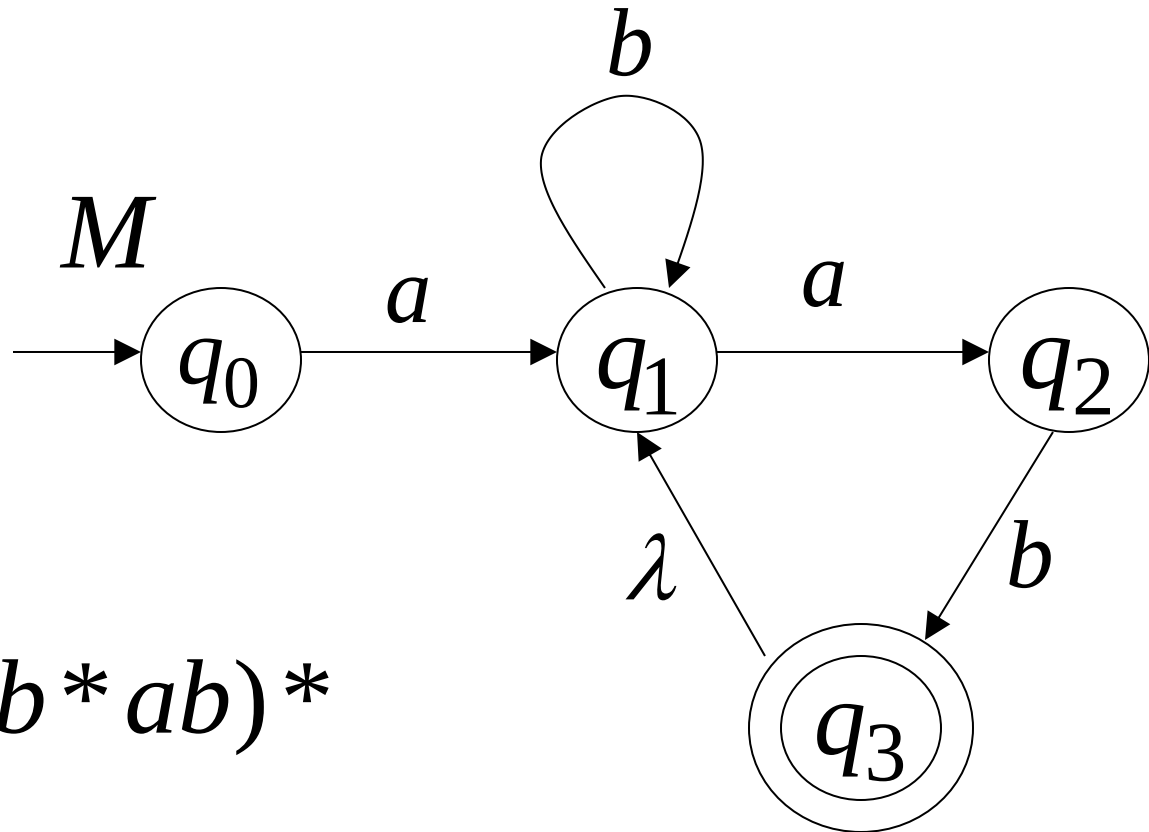
Proof idea:

Let M be the NFA with $L = L(M)$.

Construct from M a regular grammar G
such that $L(M) = L(G)$

Since L is regular
there is an NFA M such that $L = L(M)$

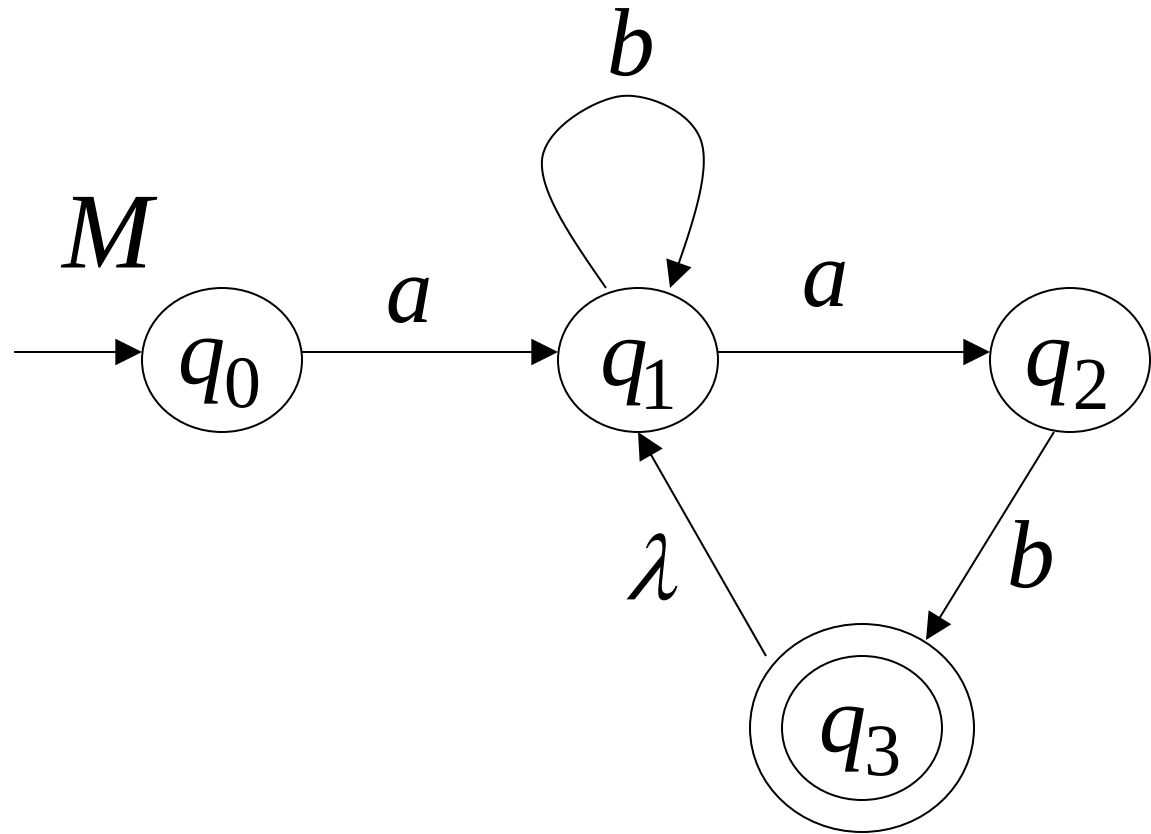
Example:



$$L = ab^*ab(b^*ab)^*$$

$$L = L(M)$$

Convert M to a right-linear grammar

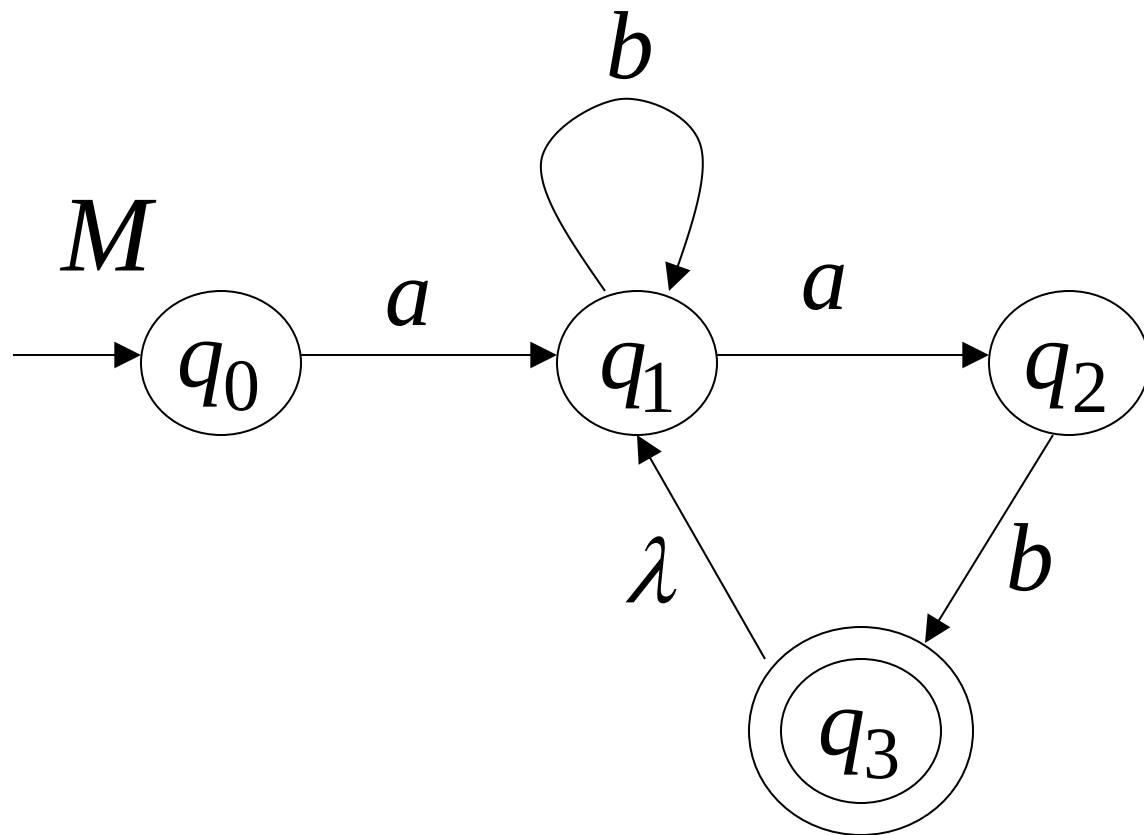


$$q_0 \rightarrow aq_1$$

$q_0 \rightarrow aq_1$

$q_1 \rightarrow bq_1$

$q_1 \rightarrow aq_2$

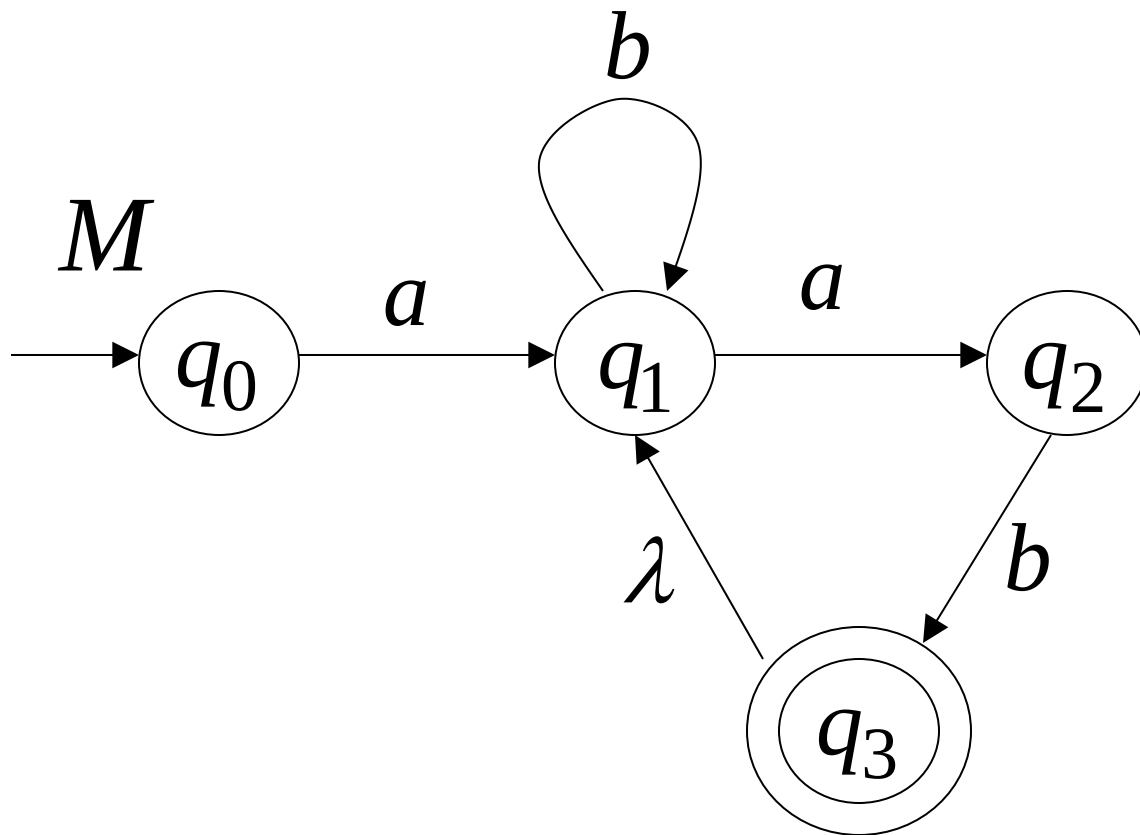


$$q_0 \rightarrow aq_1$$

$$q_1 \rightarrow bq_1$$

$$q_1 \rightarrow aq_2$$

$$q_2 \rightarrow bq_3$$



$$L(G) = L(M) = L$$

G

$$q_0 \rightarrow aq_1$$

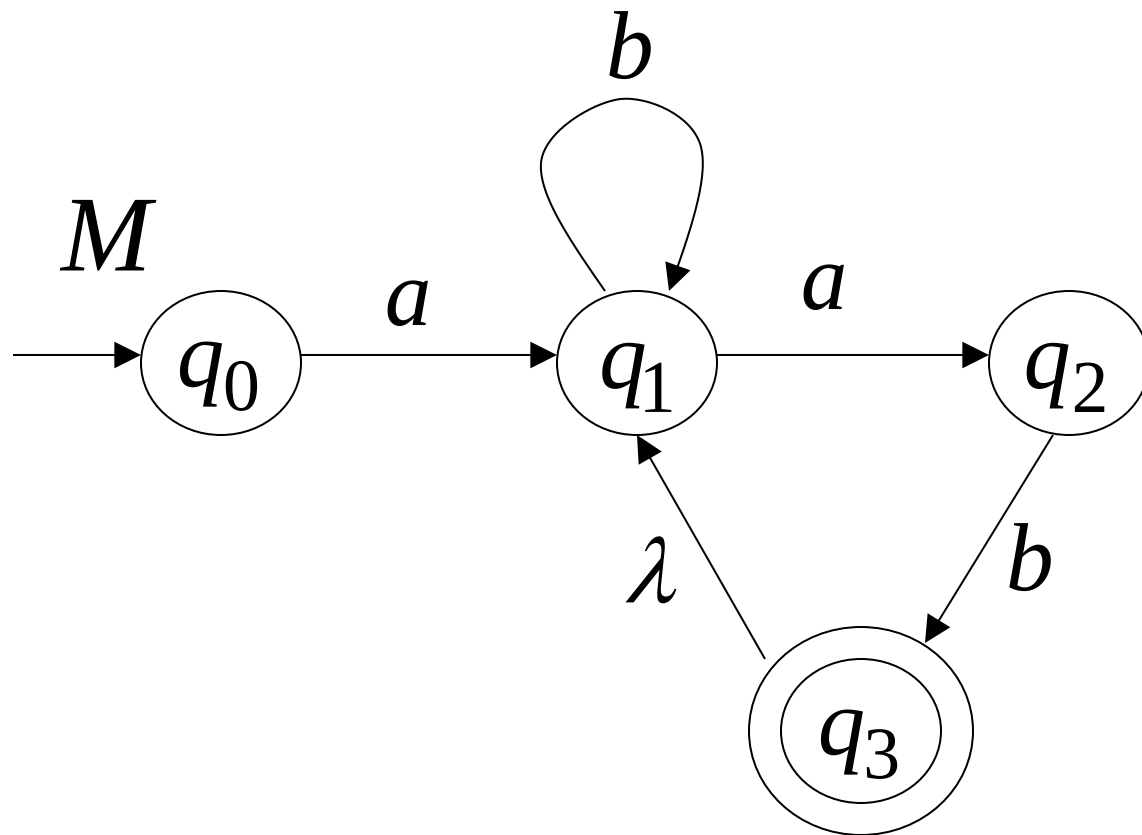
$$q_1 \rightarrow bq_1$$

$$q_1 \rightarrow aq_2$$

$$q_2 \rightarrow bq_3$$

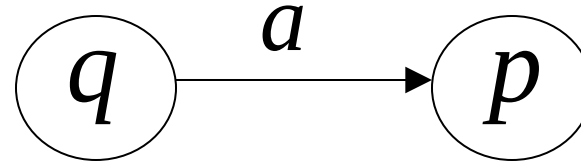
$$q_3 \rightarrow q_1$$

$$q_3 \rightarrow \lambda$$

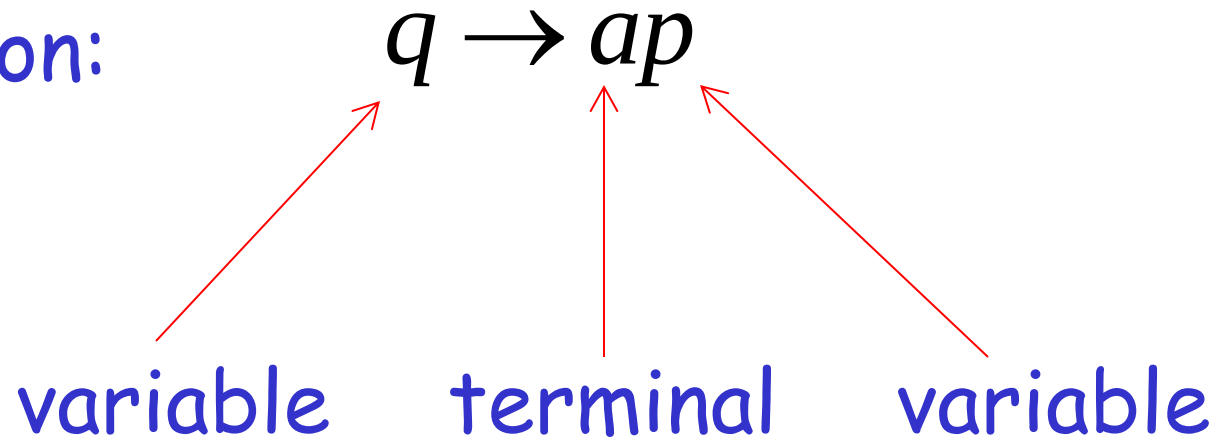


In General

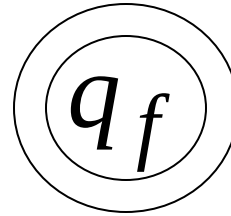
For any transition:



Add production:



For any final state:



Add production:

$$q_f \rightarrow \lambda$$

Since G is right-linear grammar

G is also a regular grammar

with $L(G) = L(M) = L$